Special Participation E: Using ChatGPT to Distinguish Similar Concepts in Notes via Embeddings

For this special participation, I designed an AI-enhanced prompt workflow using ChatGPT to help students better understand and distinguish similar concepts that appear in our lecture notes. The goal is to move beyond passive reading and create an interactive process that automatically surfaces potentially confusing concepts and then guides students through targeted explanations and practice.

The workflow proceeds in four main steps. First, I input a PDF slide file and extract the most frequently used technical terms from the lecture. Second, I generate embeddings for these candidate concepts. Third, I compute cosine similarities between these embeddings to identify pairs of concepts that are semantically close and therefore likely to be confused by students. Finally, I use ChatGPT to provide specific explanations of each similar pair and to generate a small quiz to test whether students can correctly distinguish them.

This pipeline can be applied to any lecture and shared with classmates as a reusable prompt template. It offers a structured way to focus attention on subtle conceptual differences rather than just memorizing isolated definitions. However, because ChatGPT can still hallucinate certain explanations, we should double check its answers against the notes and textbook.

Here are prompts workflow I used:

Step 0: System prompt set up

```
"""
You are an AI tutor helping a grad-level student study a SINGLE lecture from a deep learning course.
Your main goals are:
1. From the lecture notes, extract the most important technical concepts.
2. Use your internal semantic Embeddings to identify pairs of concepts that are semantically similar and easy to confuse.
3. For each confusing pair, provide clear, rigorous, and intuitive explanations that highlight the differences, including examples and common pitfalls.
4. Generate short practice questions to test whether the student can distinguish these concepts.

Constraints:
- Only use information that could reasonably be inferred from the given lecture notes and standard textbook-level knowledge.
- If you are uncertain or extrapolating beyond the notes, say so explicitly.
- Be precise with definitions, especially for math, ML, and deep learning concepts.
- Avoid statements like 'they are basically the same' unless you also explain the precise conditions under which they coincide.
"""
```

Step 1:

```
"""
I will give you pdf file from one lecture's notes.

Please:
1. OCR the text in pdf file and extract 15-30 key **technical concepts** from THIS lecture only. A 'concept' should usually be a short phrase (1-5 words), e.g. 'self-attention', 'residual connection', 'LoRA rank', 'cross-entropy loss'.
2. Ignore very generic words like 'data', 'model', 'input', unless they are part of a specific named concept.
3. For each concept, give:
   - concept name
   - a 1-2 sentence short definition in the context of this lecture
   - an approximate slide or section reference (e.g. 'Slide 5: Self-attention overview' or 'Section: Low-rank adaptation')

Return your answer as a markdown table.
"""
```

Step 2:

```
"""
Based on above concept table with brief definitions, now I want to identify **confusable concept pairs** based on semantic similarity, as if you were using embeddings and cosine similarity internally.

Please:
1. Consider semantic similarity AND the context of this specific lecture.
2. Propose 5-10 pairs (or small groups of 2-3) of concepts that are:
   - semantically similar, AND/OR
   - commonly confused by students at this level, AND/OR
   - closely related but have important differences (e.g., 'self-attention vs cross-attention', 'LoRA vs full fine-tuning').

For each pair/group, output:
- pair_id: a short identifier like P1, P2, ...
- concepts_involved: the concept names
- why_confusing: 1-2 sentences explaining why they are easy to mix up
- difference_summary: 2-4 bullet points summarizing their key conceptual differences in this lecture

Return your answer as a markdown table.
"""
```

Step 3:

```
"""
Now you are given a pair_id based on the above table, you should focus ONLY on the pair of concepts from this lecture, which are assigned to Concept A and Concept B:

- Pair_id: <<<PAIR_ID>>> MENTION: !!REPLACE THIS WITH THE ACTUAL PAIR ID

You should act like a careful TA preparing students for an exam.

Please:
1. Give a **plain-language explanation** of Concept A and Concept B separately (2-4 sentences each), assuming the reader is a grad student who already knows basic linear algebra, probability, and deep learning.
2. Create a **side-by-side comparison table** with 4-7 rows. Each row should compare a specific dimension, such as:
   - formal definition or formulation
   - typical use case in this lecture
```

```
   - intuition / mental picture
   - advantages / disadvantages
   - computational or statistical properties (if relevant)
   - how they appear in the notes or formulas
3. Give **2-3 concrete examples or scenarios** where:
   - Concept A is the appropriate tool/interpretation
   - Concept B is the appropriate tool/interpretation
   For each scenario, explain briefly why.
4. List **3-5 common student misconceptions** that confuse A and B, and correct each misconception with a short explanation.

If you need to go beyond what is literally in the notes, make reasonable textbook-level assumptions but explicitly say what is extrapolated.
"""
```

Step 4:

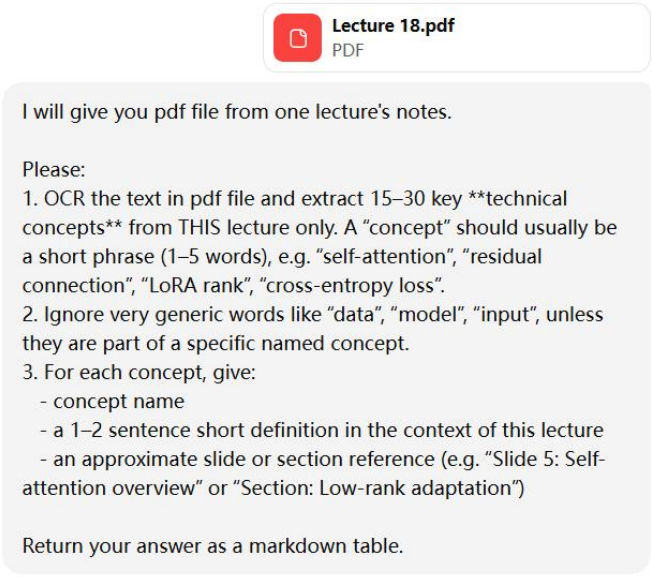
```
"""
Using the explanations and comparisons you just gave, please generate a small quiz **focused ONLY on distinguishing A and B**.

Requirements:
1. Create:
   - 3 multiple-choice questions (each with 4 options, clearly labeled A/B/C/D, one correct answer).
   - 2 short-answer questions that ask for explanations or reasoning, not just definitions.
2. For each question, provide:
   - the correct answer
   - a brief explanation (1-3 sentences) that refers back to the key differences between A and B.

Make sure the questions are non-trivial: they should require understanding the differences, not just memorizing one sentence definitions.
"""
```

Actual Practice on Lecture 18 notes:

Here is the chat Link: https://chatgpt.com/share/69348c4e-8040-8002-bf16-0ae8c39a833c

Step 1:

Prompt



Response

Below is a clean list of **25 key technical concepts** extracted from the lecture notes, based only on visible content in the PDF pages.
Each concept includes a short definition and an approximate slide reference.

✅ Concept Table (OCR-based Extraction)

Concept	Definition (context of this lecture)	Slide / Section
Encoder–Decoder Architecture	A sequence-to-sequence RNN framework where the encoder compresses the input and the decoder generates output tokens.	Slide 1
RNN Block	A recurrent module that receives the previous hidden state and current input to produce the next hidden state.	Slide 1
Bottleneck Problem	Loss of information when the entire input sequence must be encoded into a single vector before decoding.	Slide 2
Auto-regressive Generation	Decoder outputs tokens one at a time, conditioning each prediction on previous outputs.	Slide 2
Teacher Forcing	Training technique where the ground-truth token is fed to the next step instead of the model's prediction.	Slide 2
Sampling	Selecting an output token from a probability distribution during generation (instead of taking argmax).	Slide 3
Hash Table (Idea)	Proposed analogy for matching queries to stored keys, but needs to be differentiable.	Slide 4
Exact Match Retrieval	Searching for the key that exactly matches a query, but rarely works due to continuous embeddings.	Slide 4
Closest Match Retrieval	Returning the value associated with the most similar stored key when exact match fails.	Slide 4
Top-k Matches	Using only the k most similar keys when computing attention instead of all keys.	Slide 5
Weighted Average	Taking a similarity-weighted combination of the top-k keys or values to compute attention.	Slide 5
Inner Product Similarity	Using dot products between query and key vectors to measure similarity.	Slide 5
Softmax	Used to normalize similarity scores into attention weights.	Slide 5
Query Vectors (Q)	Learned projections of inputs used to match against keys and compute attention weights.	Slide 6
Key Vectors (K)	Learned projections used to measure how relevant each input token is to a given query.	Slide 6
Value Vectors (V)	Learned projections th… …rry the information retrieved by attention.	Slide 6

In this part, LLM correctly follows my instructions and give reasonable results

Step 2:

Prompt

In step 2, I ask ChatGPT to use embeddings to find confusable concept pairs, at last it find 8 pairs, which is less than what I expect. However, it clearly express the difference between pairs. Great Job!

Based on above concept table with brief definitions, now I want to identify **confusable concept pairs** based on semantic similarity, as if you were using embeddings and cosine similarity internally.

Please:
1. Consider semantic similarity AND the context of this specific lecture.
2. Propose 5–10 pairs (or small groups of 2–3) of concepts that are:
   - semantically similar, AND/OR
   - commonly confused by students at this level, AND/OR
   - closely related but have important differences (e.g., "self-attention vs cross-attention", "LoRA vs full fine-tuning").

For each pair/group, output:
- pair_id: a short identifier like P1, P2, ...
- concepts_involved: the concept names
- why_confusing: 1–2 sentences explaining why they are easy to mix up
- difference_summary: 2–4 bullet points summarizing their key conceptual differences in this lecture

Return your answer as a markdown table.

Response

✅ Confusable Concept Pairs

pair_id	concepts_involved	why_confusing	difference_summary
P1	Self-Attention vs Cross-Attention	Both use queries, keys, and values, and both appear in sequence-to-sequence models; their computational forms look very similar.	• **Self-attention:** Q,K,V from same source (encoder or decoder). • **Cross-attention:** Q from decoder, K,V from encoder. • Cross-attention allows information flow from input to output. • Self-attention models contextual relationships within the same sequence.
P2	Query vs Key vs Value	All are linear projections of the same inputs and have similar notation, which makes it unclear why three roles exist.	• **Queries:** "What I am looking for." • **Keys:** "What I contain / indexing." • **Values:** "What I return when matched." • Attention uses **Q•K** for similarity, and returns **weighted sum of V**.
P3	Inner Product Similarity vs Softmax vs Attention Weights	These appear in the same formula and students often think they are interchangeable parts.	• Inner product gives **raw similarity scores**. • Softmax converts scores into **probabilities**. • Attention weights are **normalized similarities**, used to mix values. • Removing softmax breaks gradient behavior & normalization.
P4	Scaled Dot-Product Attention vs Unscaled Attention	The only difference is a division by √d, which may seem minor, but produces large practical effects.	• Unscaled: softmax can saturate if dot products are large. • Scaled: divides by √d to stabilize gradients. • The scaling matters more when **d** is large. • Both compute **softmax(Q•K^T)** but only scaled behaves well.
P5	Encoder–Decoder Architecture vs Bottleneck Problem	Both describe constraints in seq-to-seq models, and students may think the bottleneck is a property of the architecture itself.	• Encoder–decoder: general design pattern. • Bottleneck problem: arises when **entire input is compressed into one vector**. • The issue is **capacity**, not structure. • Attention alleviates this bottleneck.
P6	Top-k Matches vs Weighted Average	Both approximate attention by focusing on nearest neighbors, but differ in how they combine them.	• Top-k: selects k nearest items. • Weighted average: mixes them proportionally to similarity. • Weighted average uses **softmax weights**. • Top-k without weights loses fine gradient information.
P7	Multi-Head Self-Attention vs Multi-Query Attention	Both reduce information into multiple streams/heads and use shared components, so the memory tradeoffs can be confusing.	• Multi-head: separate K, V, Q for each head. • Multi-query: share K and V across heads, only Q differs. • Multi-query reduces **memory and bandwidth**. • Multi-head is more flexible but costlier.
P8	Multi-Query Attention vs Multi-Query Group Attention	Names are similar and both involve "sharing", but they trade memory vs accuracy differently.	• Multi-query: **all heads share** K and V. • Group attention: heads are split into **groups**, each with its own K and V. • Grouping trades off memory vs expressiveness. • Grouping helps when many heads are used.

I think Encoder-Decoder Architecture and Bottleneck Problem are not pair

Step 3:

Prompt

Don’t forget to replace this P1 with the actual Pair_id

Now you are given a pair_id based on the above table, you should focus ONLY on the pair of concepts from this lecture, which are assigned to Concept A and Concept B:

- Pair_id: P1

You should act like a careful TA preparing students for an exam.

Please:
1. Give a **plain-language explanation** of Concept A and Concept B separately (2–4 sentences each), assuming the reader is a grad student who already knows basic linear algebra, probability, and deep learning.
2. Create a **side-by-side comparison table** with 4–7 rows. Each row should compare a specific dimension, such as:
   - formal definition or formulation
   - typical use case in this lecture
   - intuition / mental picture
   - advantages / disadvantages
   - computational or statistical properties (if relevant)
   - how they appear in the notes or formulas
3. Give **2–3 concrete examples or scenarios** where:
   - Concept A is the appropriate tool/interpretation
   - Concept B is the appropriate tool/interpretation
   For each scenario, explain briefly why.
4. List **3–5 common student misconceptions** that confuse A and B, and correct each misconception with a short explanation.

If you need to go beyond what is literally in the notes, make reasonable textbook-level assumptions but explicitly say what is extrapolated.

Response

Below is a focused explanation for **Pair P1: Self-Attention vs Cross-Attention**.

---

1) Plain-language explanations

Concept A — Self-Attention

Self-attention is a mechanism where each position in a sequence looks at *all other positions in the same sequence* to decide what is relevant. Queries, keys, and values are **all computed from the same source** (e.g., the encoder's hidden states or the decoder's own hidden states). It allows the model to capture long-range dependencies, reorder words, and share information across positions efficiently. In this lecture, self-attention replaces the bottleneck of compressing the input into a single vector by letting each token access contextual information directly.

Concept B — Cross-Attention

Cross-attention is used in sequence-to-sequence models when the model is generating output tokens and wants to consult the encoded input. Queries come from the **decoder**, while keys and values come from the **encoder**. This allows each output token to selectively retrieve relevant information from the input sequence — for instance, aligning a translated word with the corresponding word or phrase in the source sentence. Cross-attention is the bridge that connects the input sequence to the output sequence during generation.

---

2) Side-by-side comparison

Dimension	Self-Attention	Cross-Attention
Formal Definition	Q, K, V are computed from the **same** sequence (encoder or decoder).	Q comes from **decoder**, K, V come from **encoder**.
Primary Use Case in Lecture	Model internal dependencies within one sequence (e.g., a sentence).	Allow decoder to access information about the encoded input during translation.
Intuition	"Look at myself and my neighbors to understand context."	"Ask the encoder what part of the input is relevant to generate this output token."
Information Flow Direction	Horizontal: within the same time axis.	Vertical: from input (encoder) to output (decoder).
When Evaluated	During encoding and sometimes during decoding.	Only during decoding (when generating outputs).
Advantages	Captures long-range dependencies in a single pass; no bottleneck.	Enables **alignment** between source and target sequences; improves translation accuracy.
Disadvantages	Cannot inject new information from the input once encoding is done.	More computational cost when encoder representations are long.
How it appears in notes	"Keys + values + queries from decoder (or encoder)." ↓	"Keys + values from encoder, queries from decoder."

3) Examples / Scenarios

Scenario 1 — Word reordering in a sentence

You want to understand how the word "book" in the sentence relates to "the" and "give."

- **Use self-attention.**
  All tokens come from the same sentence. Self-attention can capture that "give me the book" has a structure where "the book" is a noun phrase regardless of word positions.

Scenario 2 — Translation

You are translating "Give me the book" to Hindi.

- **Use cross-attention.**
  Each output token in Hindi (e.g., "किताब") must consult the source sentence's representations to align the correct meaning.

Scenario 3 — Decoder refinement

While generating the next token, the decoder also wants to consider previous generated tokens.

- **Use self-attention inside the decoder**.
- **Use cross-attention to look at encoder**.

Most transformer decoders do **both** at each layer:

1. self-attention (intra-decoder)
2. cross-attention (decoder → encoder)

---

4) Common student misconceptions

Misconception 1 — "Self-attention and cross-attention use different formulas."

Correction:
The formulas are almost identical. Both compute $QK^\top$, then softmax, then a weighted sum of V.
The difference is **where Q, K, V come from**.

Misconception 2 — "Cross-attention replaces self-attention in the decoder."

Correction:
A transformer decoder has **both**:

- self-attention (to look at previously generated tokens),
- cross-attention (to look at encoder outputs).

They address different needs.

Misconception 3 — "Self-attention cannot access encoder information."

Correction:
Correct. Self-attention only mixes information within the same sequence.
To access encoder information, you need **cross-attention**.

Misconception 4 — "Cross-attention is only useful in translation."

Correction (slightly extrapolated):
Cross-attention is used in any seq-to-seq or retrieval-augmented setting:

- summarization,
- image captioning,
- multimodal models (vision → text).

Misconception 5 — "Cross-attention is more powerful than self-attention."

Correction:
Neither is "more powerful." They serve **different purposes**:

- self-attention: builds context within a sequence,
- cross-attention: injects external information.

Step 4

Prompt

I love this part because it can test whether I have digest above knowledge

Using the explanations and comparisons you just gave, please generate a small quiz **focused ONLY on distinguishing A and B**.

Requirements:
1. Create:
  - 3 multiple-choice questions (each with 4 options, clearly labeled A/B/C/D, one correct answer).
  - 2 short-answer questions that ask for explanations or reasoning, not just definitions.
2. For each question, provide:
  - the correct answer
  - a brief explanation (1–3 sentences) that refers back to the key differences between A and B.

Make sure the questions are non-trivial: they should require understanding the differences, not just memorizing one sentence definitions.

Response

✅ Multiple-Choice Questions

MCQ 1

In a transformer decoder layer, which statement is correct?

A. Self-attention retrieves information from the encoder hidden states.
B. Cross-attention retrieves information from the encoder hidden states.
C. Both self-attention and cross-attention retrieve information from encoder hidden states.
D. Neither self-attention nor cross-attention retrieve information from encoder hidden states.

Correct answer: B

Explanation:
Cross-attention uses **queries from the decoder** and **keys/values from the encoder**, allowing access to encoder information. Self-attention only mixes information from **within the same sequence**.

MCQ 2

Which of the following best describes the purpose of self-attention in the decoder?

A. To align output tokens with relevant parts of the input sequence.
B. To allow the decoder to consult the encoder for new information.
C. To model dependencies among the previously generated output tokens.
D. To replace cross-attention entirely.

Correct answer: C

Explanation:
Self-attention in the decoder lets each output token look at **previous output tokens** to maintain consistency. Alignment with input happens through **cross-attention**, not self-attention.

MCQ 3

Which situation most clearly requires cross-attention rather than self-attention?

A. Computing the relationship between words inside a single sentence.
B. Modeling how "book" relates to "the" within an English clause.
C. Translating "book" into Hindi when generating a new token.
D. Predicting the next character within a long string of text.

Correct answer: C

Explanation:
Translation requires consulting the **encoded source sentence**, so the decoder uses **cross-attention**. Self-attention only handles relationships **within a single sequence**.

✅ Short-Answer Questions

SA 1

Explain why a transformer decoder typically contains **both** self-attention and cross-attention. What distinct roles do they play?

Correct answer explanation:
Self-attention allows the decoder to look at **previously generated tokens** to maintain coherence and grammar. Cross-attention allows the decoder to **consult the encoder output** to retrieve relevant information from the input sequence (e.g., alignment in translation). They solve **different problems**, so both are needed.

SA 2

Suppose a model is performing summarization. During generation, it needs to ensure that each output sentence accurately reflects the source text. Which attention mechanism provides access to the source text, and why?

Correct answer explanation:
Cross-attention provides access to the source text because **queries come from the decoder** and keys/values come from the **encoder representation of the source document**. Self-attention cannot inject new information from the source; it only mixes information within the decoder's own outputs.